Universidad CENFOTEC

Escuela de Ingeniería del Software
Código del curso: SOFT-01
Nombre del curso: Principios de Programación 1
Sección: SCV6
Periodo: C3-2026
Docente: Verónica Mora Lezcano

Misión Órbita

Simulador de exploración espacial en Python

Primer avance del proyecto: Preparación de la misión

Integrantes	Bryner Moraga Gomez
Repositorio Git	https://github.com/brynermg/Proyecto_Principios_Programacion.git
Fecha de entrega	14 de octubre de 2026

Descripción general

Este avance implementa cuatro algoritmos independientes que representan la preparación de una misión, siguiendo el esquema entrada → procesamiento → decisión → salida. Cada ejercicio se presenta con su tabla de variables, diagrama de flujo, código fuente y casos de prueba con evidencia de ejecución en consola. Todos los programas respetan las restricciones técnicas: solo consola, sin ciclos, colecciones, funciones propias, módulos, archivos, POO, condicionales anidadas ni elif.

Ejercicio	Archivo	Estructura de control
Cálculo de recursos necesarios para una misión	avance1/ejercicio1_recursos.py	secuencial
Verificación de combustible	avance1/ejercicio2_combustible.py	condicional simple (if)
Verificación del soporte para la tripulación	avance1/ejercicio3_soporte_tripulacion.py	condicional doble (if/else) con condición compuesta
Autorización inicial de lanzamiento	avance1/ejercicio4_autorizacion.py	condicional doble (if/else) sin anidación, con condición compuesta de los cuatro recursos

Notación de los diagramas: terminal redondeado (Inicio/Fin), rectángulo (declaración de constantes, en morado), paralelogramo (entrada “Leer” / salida “Mostrar”), rectángulo (proceso o asignación), rombo (decisión con salidas Sí/No), círculo con letra (conector entre columnas) y punto negro (unión de las ramas). Los diagramas se elaboraron en draw.io y los archivos .drawio se incluyen en el repositorio para su edición.

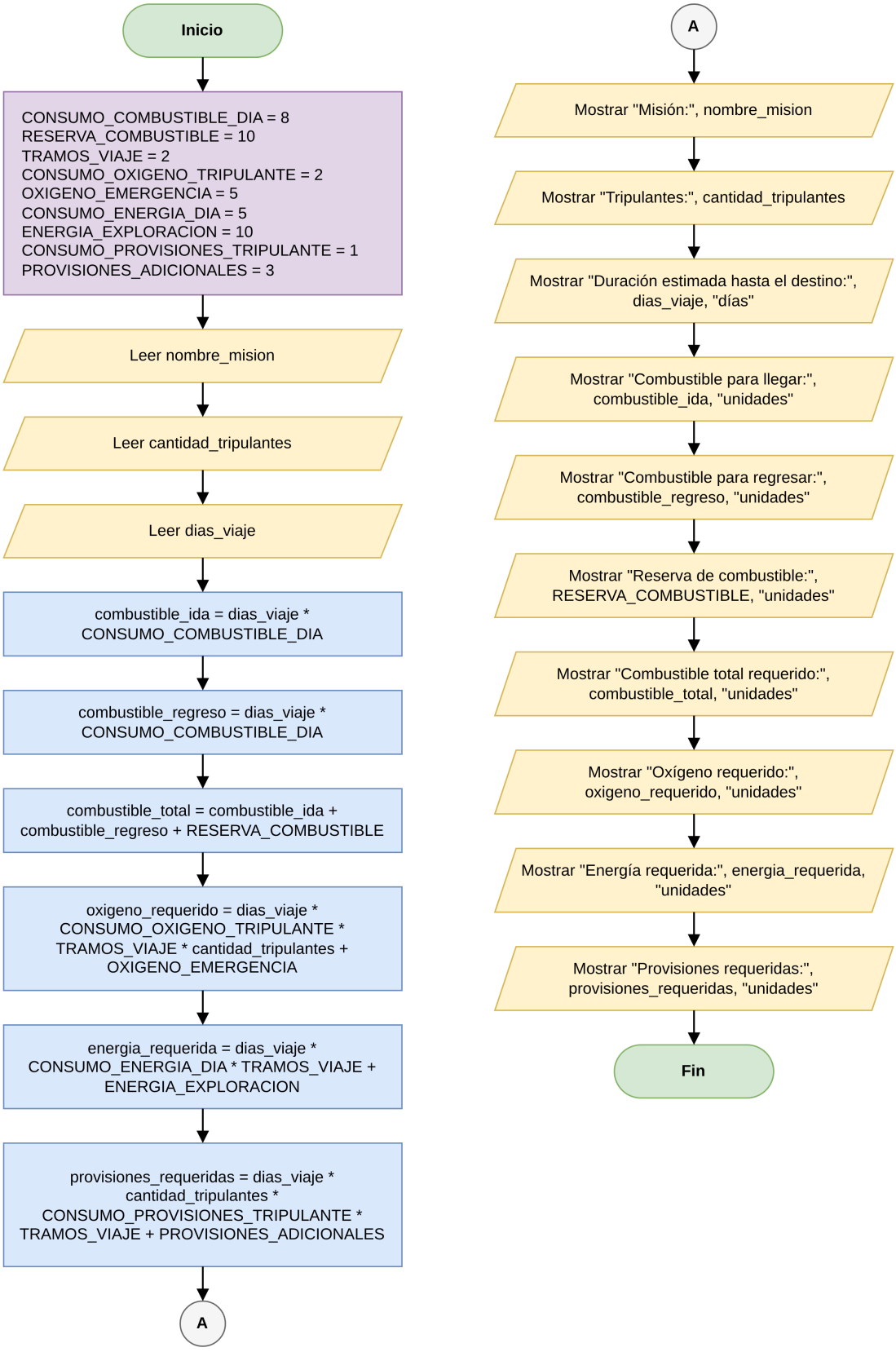
Ejercicio 1: Cálculo de recursos necesarios para una misión

Estructura de control: secuencial. **Archivo:** avance1/ejercicio1_recursos.py

Tabla de variables

Nombre	Tipo de dato	Descripción	Clasificación	Valor inicial / Restricción
CONSUMO_COMBUSTIBLE_DIA	int	Combustible consumido por día de viaje	Constante	8
RESERVA_COMBUSTIBLE	int	Reserva obligatoria de combustible para emergencias	Constante	10
TRAMOS_VIAJE	int	Cantidad de tramos del viaje (ida y regreso)	Constante	2
CONSUMO_OXIGENO_TRIPULANTE	int	Oxígeno por tripulante por día	Constante	2
OXIGENO_EMERGENCIA	int	Oxígeno adicional de emergencia	Constante	5
CONSUMO_ENERGIA_DIA	int	Energía consumida por día de viaje	Constante	5
ENERGIA_EXPLORACION	int	Energía para operaciones de exploración	Constante	10
CONSUMO_PROVISIONES_TRIPULANTE	int	Provisiones por tripulante por día	Constante	1
PROVISIONES_ADICIONALES	int	Provisiones extra de la misión	Constante	3
nombre_mision	str	Nombre de la misión	Entrada	Texto no vacío
cantidad_tripulantes	int	Cantidad de tripulantes	Entrada	Entero positivo (> 0)
dias_viaje	int	Días estimados para llegar al destino	Entrada	Entero positivo (> 0)
combustible_ida	int	Combustible para llegar al destino	Salida	dias_viaje × 8
combustible_regreso	int	Combustible para regresar	Salida	dias_viaje × 8
combustible_total	int	Ida + regreso + reserva	Salida	Calculado
oxigeno_requerido	int	Oxígeno total para la tripulación (ida y regreso)	Salida	Calculado
energia_requerida	int	Energía total de la nave	Salida	Calculado
provisiones_requeridas	int	Provisiones totales de la tripulación	Salida	Calculado

Diagrama de flujo — Ejercicio 1



Elaborado en draw.io — archivo editable: documentacion/diagramas/diagrama_ej1.drawio

Código fuente — ejercicio1_recursos.py

```
# ============================================================
# Archivo: ejercicio1_recursos.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Bryner Moraga Gomez
# Fecha: 12/10/2026    Versión: 1.0
# Descripción: Calcula los recursos necesarios (combustible,
#              oxígeno, energía y provisiones) para una misión.
#              Estructura de control: secuencial.
# ============================================================

# ---------- Declaración de constantes ----------
# Valores fijos de la consigna; se declaran aquí para evitar
# "números mágicos" dentro de las fórmulas.
CONSUMO_COMBUSTIBLE_DIA = 8        # unidades de combustible por día de viaje
RESERVA_COMBUSTIBLE = 10           # reserva obligatoria para emergencias
TRAMOS_VIAJE = 2                   # ida + regreso (misma duración)
CONSUMO_OXIGENO_TRIPULANTE = 2     # oxígeno por tripulante por día
OXIGENO_EMERGENCIA = 5             # oxígeno adicional de emergencia
CONSUMO_ENERGIA_DIA = 5            # energía por día de viaje
ENERGIA_EXPLORACION = 10           # energía para operaciones de exploración
CONSUMO_PROVISIONES_TRIPULANTE = 1 # provisiones por tripulante por día
PROVISIONES_ADICIONALES = 3        # provisiones extra

# ---------- Entrada de datos ----------
nombre_mision = input("Ingrese el nombre de la misión: ")
cantidad_tripulantes = int(input("Ingrese la cantidad de tripulantes: "))
dias_viaje = int(input("Ingrese los días estimados para llegar al destino: "))

# ---------- Procesamiento ----------
# El regreso dura lo mismo que la ida, por eso ambos tramos
# usan los mismos días y el mismo consumo diario.
combustible_ida = dias_viaje * CONSUMO_COMBUSTIBLE_DIA
combustible_regreso = dias_viaje * CONSUMO_COMBUSTIBLE_DIA

# La reserva se suma porque la nave no debe llegar justa al regreso.
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

# Oxígeno y provisiones dependen de la tripulación y de ambos tramos.
oxigeno_requerido = (dias_viaje * CONSUMO_OXIGENO_TRIPULANTE * TRAMOS_VIAJE
                     * cantidad_tripulantes + OXIGENO_EMERGENCIA)

# La energía depende solo de los días (la nave), no de la tripulación.
energia_requerida = (dias_viaje * CONSUMO_ENERGIA_DIA * TRAMOS_VIAJE
                     + ENERGIA_EXPLORACION)

provisiones_requeridas = (dias_viaje * cantidad_tripulantes
                          * CONSUMO_PROVISIONES_TRIPULANTE * TRAMOS_VIAJE
                          + PROVISIONES_ADICIONALES)

# ---------- Salida de resultados ----------
print()
print("Misión:", nombre_mision)
print("Tripulantes:", cantidad_tripulantes)
print("Duración estimada hasta el destino:", dias_viaje, "días")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Reserva de combustible:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Oxígeno requerido:", oxigeno_requerido, "unidades")
print("Energía requerida:", energia_requerida, "unidades")
print("Provisiones requeridas:", provisiones_requeridas, "unidades")
```

Casos de prueba — Ejercicio 1

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-01	Ejercicio 1	Ejemplo de la consigna (3 tripulantes, 5 días)	nombre_mision = Explorador I cantidad_tripulantes = 3 dias_viaje = 5	Ida = 40, Regreso = 40, Reserva = 10 Total = 90 Oxígeno = 65 Energía = 60 Provisiones = 33	Combustible para llegar: 40 unidades Combustible para regresar: 40 unidades Combustible total requerido: 90 unidades Oxígeno requerido: 65 unidades Energía requerida: 60 unidades Provisiones requeridas: 33 unidades	Exitoso
CP-02	Ejercicio 1	Valores mínimos (1 tripulante, 1 día)	nombre_mision = Prueba Mínima cantidad_tripulantes = 1 dias_viaje = 1	Ida = 8, Regreso = 8, Reserva = 10 Total = 26 Oxígeno = 9 Energía = 20 Provisiones = 5	Combustible para llegar: 8 unidades Combustible para regresar: 8 unidades Combustible total requerido: 26 unidades Oxígeno requerido: 9 unidades Energía requerida: 20 unidades Provisiones requeridas: 5 unidades	Exitoso
CP-03	Ejercicio 1	Misión larga (4 tripulantes, 10 días)	nombre_mision = Andrómeda cantidad_tripulantes = 4 dias_viaje = 10	Ida = 80, Regreso = 80, Reserva = 10 Total = 170 Oxígeno = 165 Energía = 110 Provisiones = 83	Combustible para llegar: 80 unidades Combustible para regresar: 80 unidades Combustible total requerido: 170 unidades Oxígeno requerido: 165 unidades Energía requerida: 110 unidades Provisiones requeridas: 83 unidades	Exitoso

Evidencia de ejecución

```
Terminal — mision-orbita/avance1
$ python3 ejercicio1_recursos.py
Ingrese el nombre de la misión: Explorador I
Ingrese la cantidad de tripulantes: 3
Ingrese los días estimados para llegar al destino: 5

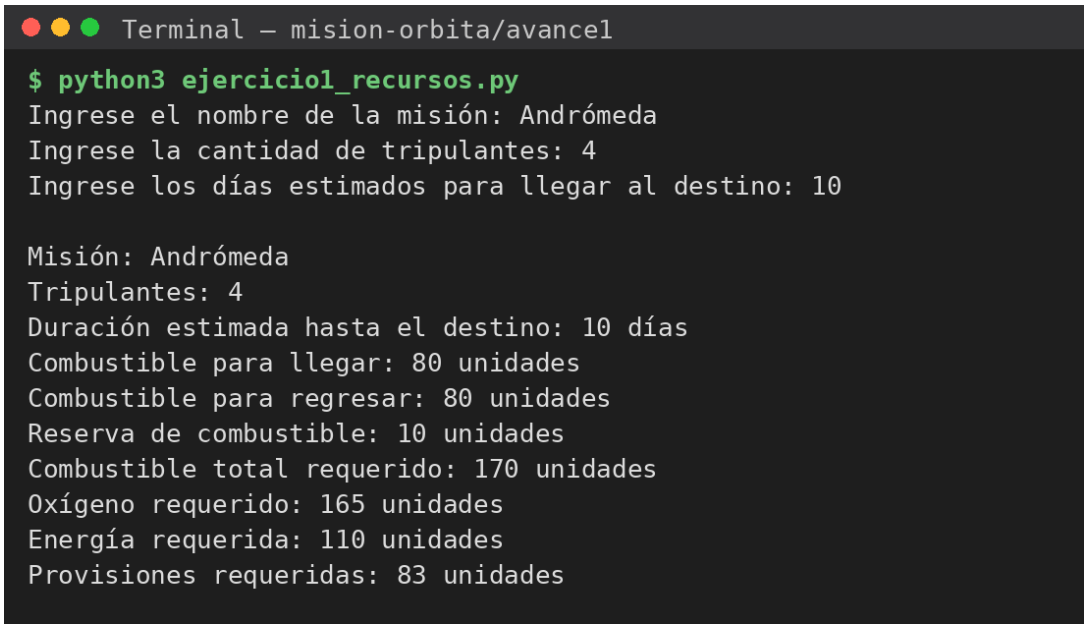
Misión: Explorador I
Tripulantes: 3
Duración estimada hasta el destino: 5 días
Combustible para llegar: 40 unidades
Combustible para regresar: 40 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 90 unidades
Oxígeno requerido: 65 unidades
Energía requerida: 60 unidades
Provisiones requeridas: 33 unidades
```

CP-01 — Ejemplo de la consigna (3 tripulantes, 5 días)

```
Terminal — mision-orbita/avance1
$ python3 ejercicio1_recursos.py
Ingrese el nombre de la misión: Prueba Mínima
Ingrese la cantidad de tripulantes: 1
Ingrese los días estimados para llegar al destino: 1

Misión: Prueba Mínima
Tripulantes: 1
Duración estimada hasta el destino: 1 días
Combustible para llegar: 8 unidades
Combustible para regresar: 8 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 26 unidades
Oxígeno requerido: 9 unidades
Energía requerida: 20 unidades
Provisiones requeridas: 5 unidades
```

CP-02 — Valores mínimos (1 tripulante, 1 día)



CP-03 — Misión larga (4 tripulantes, 10 días)

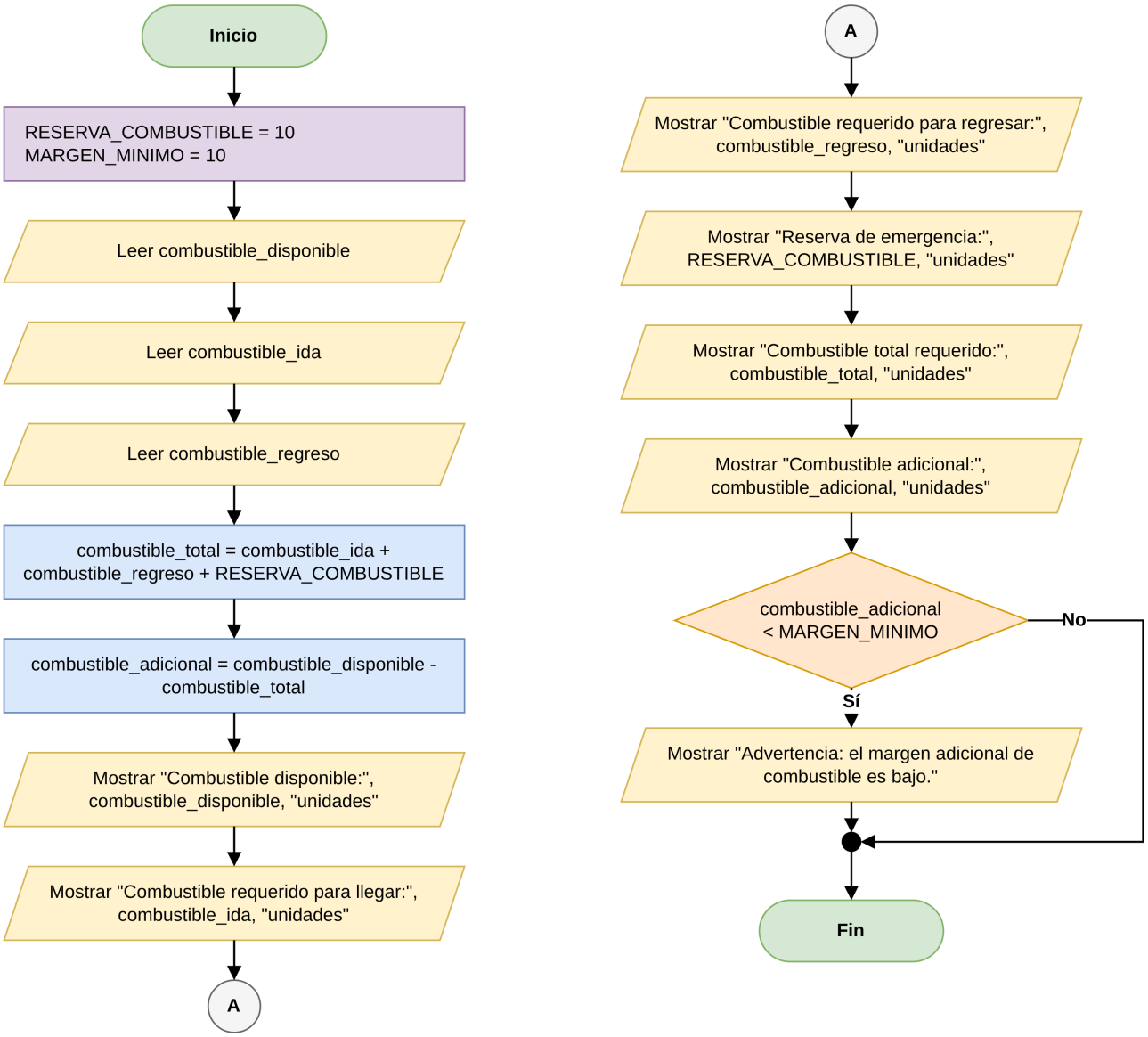
Ejercicio 2: Verificación de combustible

Estructura de control: condicional simple (if). **Archivo:** avance1/ejercicio2_combustible.py

Tabla de variables

Nombre	Tipo de dato	Descripción	Clasificación	Valor inicial / Restricción
RESERVA_COMBUSTIBLE	int	Reserva obligatoria de combustible para emergencias	Constante	10
MARGEN_MINIMO	int	Margen adicional mínimo; por debajo se advierte	Constante	10
combustible_disponible	int	Combustible con que cuenta la nave	Entrada	Entero positivo (> 0)
combustible_ida	int	Combustible requerido para la ida	Entrada	Entero positivo (> 0)
combustible_regreso	int	Combustible requerido para el regreso	Entrada	Entero positivo (> 0)
combustible_total	int	Ida + regreso + reserva	Salida	Calculado
combustible_adicional	int	Sobrante: disponible − total (puede ser negativo); se usa en la decisión	Salida	Calculado

Diagrama de flujo — Ejercicio 2



Elaborado en draw.io — archivo editable: documentacion/diagramas/diagrama_ej2.drawio

Código fuente — ejercicio2_combustible.py

```
# ============================================================
# Archivo: ejercicio2_combustible.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Bryner Moraga Gomez
# Fecha: 12/10/2026    Versión: 1.0
# Descripción: Verifica si la nave posee suficiente combustible
#              para ida, regreso y reserva, y advierte cuando el
#              margen adicional es bajo.
#              Estructura de control: condicional simple (if).
# ============================================================

# ---------- Declaración de constantes ----------
RESERVA_COMBUSTIBLE = 10    # reserva obligatoria para emergencias
MARGEN_MINIMO = 10          # por debajo de este margen se advierte

# ---------- Entrada de datos ----------
combustible_disponible = int(input("Ingrese el combustible disponible: "))
combustible_ida = int(input("Ingrese el combustible requerido para la ida: "))
combustible_regreso = int(input("Ingrese el combustible requerido para el regreso: "))

# ---------- Procesamiento ----------
# El total incluye el regreso y la reserva: llegar no es suficiente.
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

# El adicional es lo que sobra después de cubrir todo lo requerido;
# puede ser negativo si el combustible no alcanza.
combustible_adicional = combustible_disponible - combustible_total

# ---------- Salida de resultados (siempre se muestran) ----------
print()
print("Combustible disponible:", combustible_disponible, "unidades")
print("Combustible requerido para llegar:", combustible_ida, "unidades")
print("Combustible requerido para regresar:", combustible_regreso, "unidades")
print("Reserva de emergencia:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Combustible adicional:", combustible_adicional, "unidades")

# ---------- Decisión ----------
# Condicional simple: solo se actúa cuando el margen es menor que
# el mínimo (estrictamente menor; un margen de 10 no genera advertencia).
if combustible_adicional < MARGEN_MINIMO:
    print("Advertencia: el margen adicional de combustible es bajo.")
```

Casos de prueba — Ejercicio 2

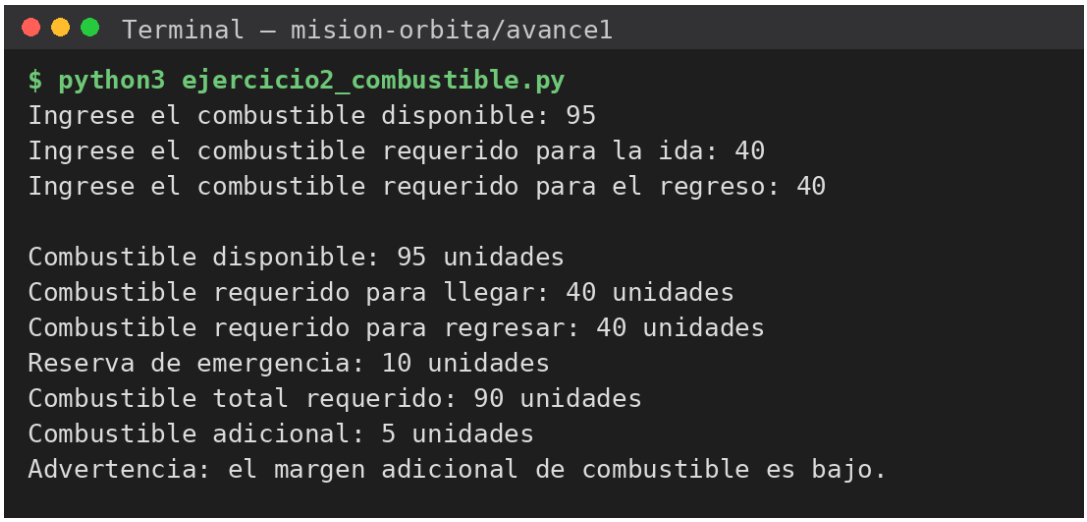
ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-04	Ejercicio 2	Combustible suficiente con margen amplio	combustible_disponible = 200 combustible_ida = 40 combustible_regreso = 40	Total = 90 Adicional = 110 No muestra advertencia	Combustible total requerido: 90 unidades Combustible adicional: 110 unidades (sin advertencia)	Exitoso
CP-05	Ejercicio 2	Suficiente para ida y regreso pero con poco margen	combustible_disponible = 95 combustible_ida = 40 combustible_regreso = 40	Total = 90 Adicional = 5 Muestra advertencia	Combustible total requerido: 90 unidades Combustible adicional: 5 unidades Advertencia: el margen adicional de combustible es bajo.	Exitoso
CP-06	Ejercicio 2	Caso límite: adicional exactamente igual a 10	combustible_disponible = 100 combustible_ida = 40 combustible_regreso = 40	Total = 90 Adicional = 10 No muestra advertencia (10 no es < 10)	Combustible total requerido: 90 unidades Combustible adicional: 10 unidades (sin advertencia)	Exitoso
CP-07	Ejercicio 2	Caso límite: disponible exactamente igual al total	combustible_disponible = 90 combustible_ida = 40 combustible_regreso = 40	Total = 90 Adicional = 0 Muestra advertencia	Combustible total requerido: 90 unidades Combustible adicional: 0 unidades Advertencia: el margen adicional de combustible es bajo.	Exitoso
CP-08	Ejercicio 2	Combustible solo para llegar, no para regresar	combustible_disponible = 50 combustible_ida = 40 combustible_regreso = 40	Total = 90 Adicional = -40 Muestra advertencia	Combustible total requerido: 90 unidades Combustible adicional: -40 unidades Advertencia: el margen adicional de combustible es bajo.	Exitoso

Evidencia de ejecución

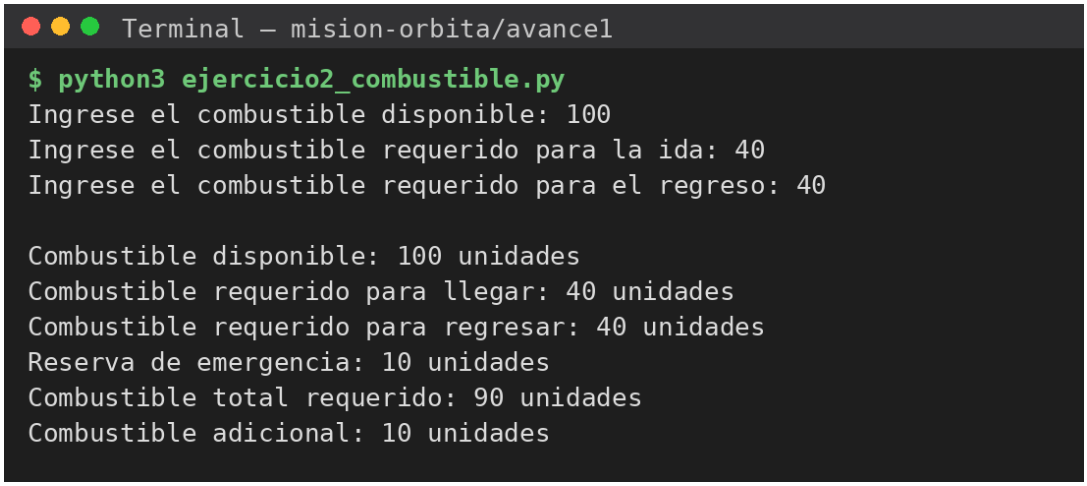
```
Terminal — mision-orbita/avance1
$ python3 ejercicio2_combustible.py
Ingrese el combustible disponible: 200
Ingrese el combustible requerido para la ida: 40
Ingrese el combustible requerido para el regreso: 40

Combustible disponible: 200 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional: 110 unidades
```

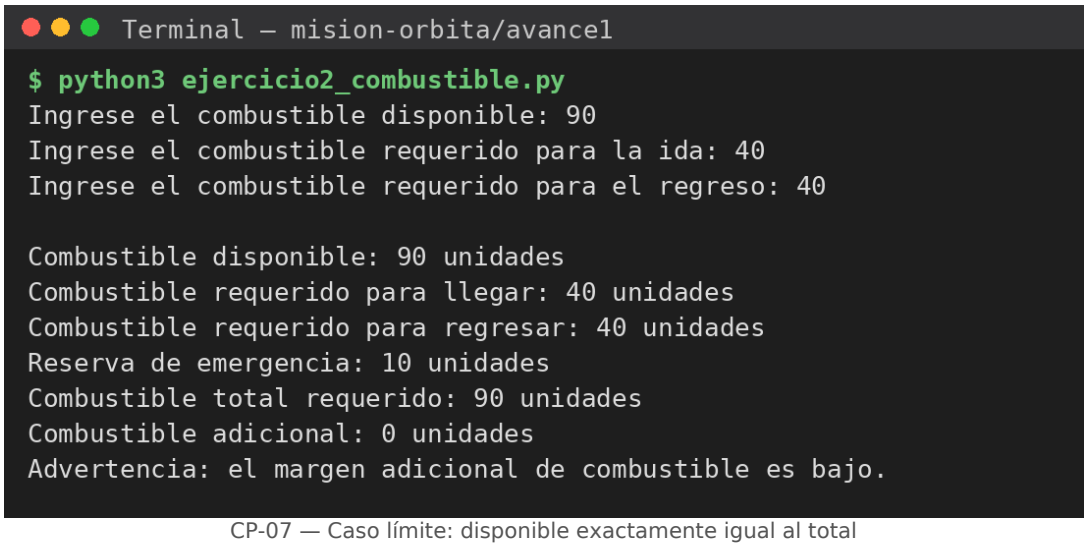
CP-04 — Combustible suficiente con margen amplio



CP-05 — Suficiente para ida y regreso pero con poco margen



CP-06 — Caso límite: adicional exactamente igual a 10



CP-07 — Caso límite: disponible exactamente igual al total

```
Terminal — mision-orbita/avance1
$ python3 ejercicio2_combustible.py
Ingrese el combustible disponible: 50
Ingrese el combustible requerido para la ida: 40
Ingrese el combustible requerido para el regreso: 40

Combustible disponible: 50 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional: -40 unidades
Advertencia: el margen adicional de combustible es bajo.
```

CP-08 — Combustible solo para llegar, no para regresar

Ejercicio 3: Verificación del soporte para la tripulación

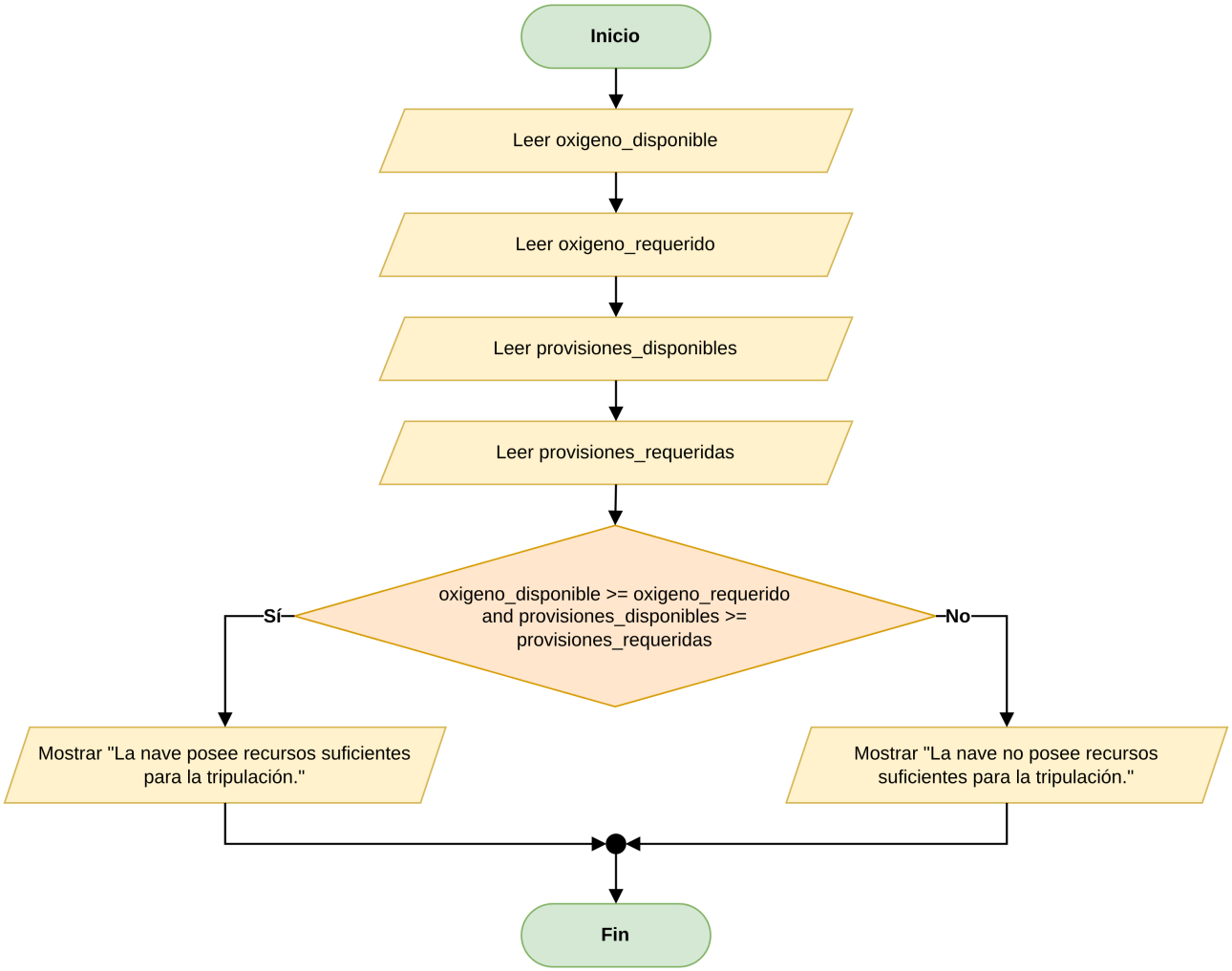
Estructura de control: condicional doble (if/else) con condición compuesta. **Archivo:** avance1/ejercicio3_soporte_tripulacion.py

Tabla de variables

Nombre	Tipo de dato	Descripción	Clasificación	Valor inicial / Restricción
oxigeno_disponible	int	Oxígeno con que cuenta la nave	Entrada	Entero positivo (> 0)
oxigeno_requerido	int	Oxígeno necesario para la tripulación	Entrada	Entero positivo (> 0)
provisiones_disponibles	int	Provisiones con que cuenta la nave	Entrada	Entero positivo (> 0)
provisiones_requeridas	int	Provisiones necesarias para la tripulación	Entrada	Entero positivo (> 0)

Los mensajes "La nave posee / no posee recursos suficientes para la tripulación." son literales de salida (str) mostrados según la decisión; no se almacenan en variables.

Diagrama de flujo — Ejercicio 3



Elaborado en draw.io — archivo editable: documentacion/diagramas/diagrama_ej3.drawio

Código fuente — ejercicio3_soporte_tripulacion.py

```
# ============================================================
# Archivo: ejercicio3_soporte_tripulacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Bryner Moraga Gomez
# Fecha: 12/10/2026    Versión: 1.0
# Descripción: Verifica si la nave tiene oxígeno y provisiones
#              suficientes para mantener a la tripulación.
#              Estructura de control: condicional doble (if/else).
# ============================================================

# ---------- Declaración de constantes ----------
# Este ejercicio no requiere constantes: solo compara valores
# disponibles contra valores requeridos ingresados por el usuario.

# ---------- Entrada de datos ----------
oxigeno_disponible = int(input("Ingrese el oxígeno disponible: "))
oxigeno_requerido = int(input("Ingrese el oxígeno requerido: "))
provisiones_disponibles = int(input("Ingrese las provisiones disponibles: "))
provisiones_requeridas = int(input("Ingrese las provisiones requeridas: "))

# ---------- Decisión y salida de resultados ----------
# Se usa "and" porque la tripulación necesita AMBOS recursos;
# ">=" porque tener exactamente lo requerido sí es suficiente.
# No se indica cuál recurso falla, según la consigna.
print()
if oxigeno_disponible >= oxigeno_requerido and provisiones_disponibles >= provisiones_requeridas:
    print("La nave posee recursos suficientes para la tripulación.")
else:
    print("La nave no posee recursos suficientes para la tripulación.")
```

Casos de prueba — Ejercicio 3

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-09	Ejercicio 3	Ambos recursos suficientes	oxigeno_disponible = 100 oxigeno_requerido = 65 provisiones_disponibles = 50 provisiones_requeridas = 33	Recursos suficientes	La nave posee recursos suficientes para la tripulación.	Exitoso
CP-10	Ejercicio 3	Caso límite: ambos exactamente iguales al requerido	oxigeno_disponible = 65 oxigeno_requerido = 65 provisiones_disponibles = 33 provisiones_requeridas = 33	Recursos suficientes (>= incluye la igualdad)	La nave posee recursos suficientes para la tripulación.	Exitoso
CP-11	Ejercicio 3	Oxígeno insuficiente, provisiones suficientes	oxigeno_disponible = 60 oxigeno_requerido = 65 provisiones_disponibles = 50 provisiones_requeridas = 33	Recursos no suficientes (sin indicar cuál)	La nave no posee recursos suficientes para la tripulación.	Exitoso
CP-12	Ejercicio 3	Provisiones insuficientes, oxígeno suficiente	oxigeno_disponible = 100 oxigeno_requerido = 65 provisiones_disponibles = 30 provisiones_requeridas = 33	Recursos no suficientes (sin indicar cuál)	La nave no posee recursos suficientes para la tripulación.	Exitoso
CP-13	Ejercicio 3	Ambos recursos insuficientes	oxigeno_disponible = 10 oxigeno_requerido = 65 provisiones_disponibles = 10 provisiones_requeridas = 33	Recursos no suficientes	La nave no posee recursos suficientes para la tripulación.	Exitoso

Evidencia de ejecución

```
Terminal — mision-orbita/avance1
$ python3 ejercicio3_soporte_tripulacion.py
Ingrese el oxígeno disponible: 100
Ingrese el oxígeno requerido: 65
Ingrese las provisiones disponibles: 50
Ingrese las provisiones requeridas: 33

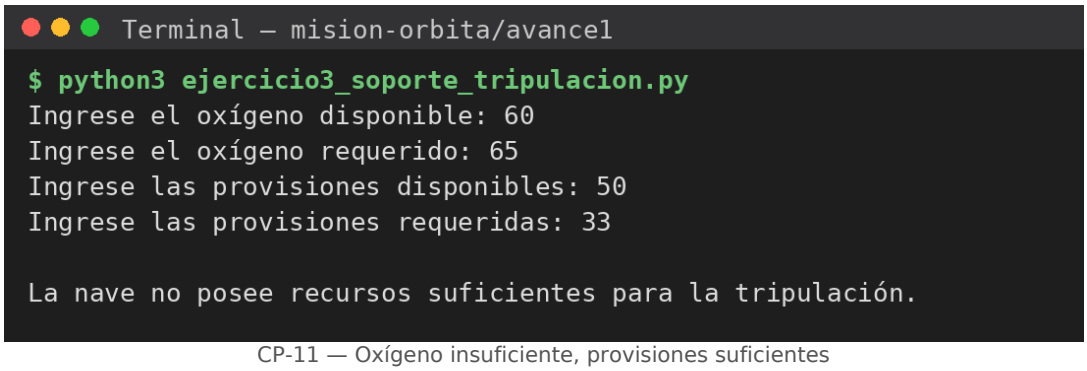
La nave posee recursos suficientes para la tripulación.
```

CP-09 — Ambos recursos suficientes

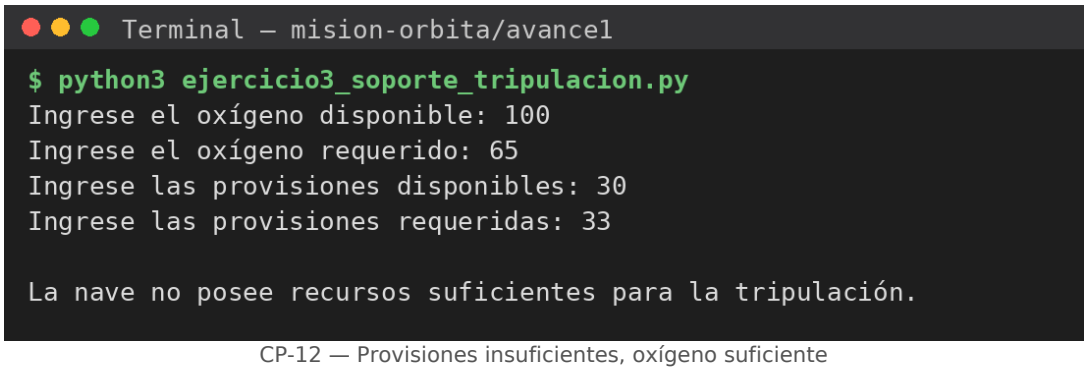
```
Terminal — mision-orbita/avance1
$ python3 ejercicio3_soporte_tripulacion.py
Ingrese el oxígeno disponible: 65
Ingrese el oxígeno requerido: 65
Ingrese las provisiones disponibles: 33
Ingrese las provisiones requeridas: 33

La nave posee recursos suficientes para la tripulación.
```

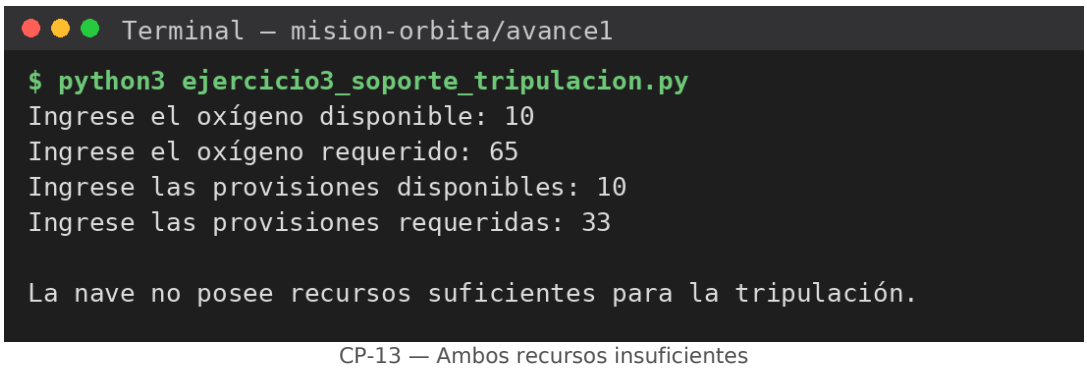
CP-10 — Caso límite: ambos exactamente iguales al requerido



CP-11 — Oxígeno insuficiente, provisiones suficientes



CP-12 — Provisiones insuficientes, oxígeno suficiente



CP-13 — Ambos recursos insuficientes

Ejercicio 4: Autorización inicial de lanzamiento

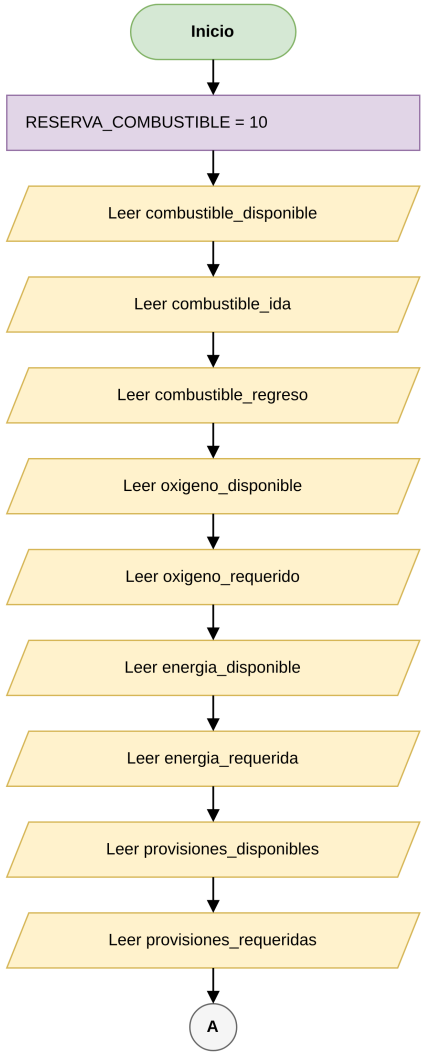
Estructura de control: condicional doble (if/else) sin anidación, con condición compuesta de los cuatro recursos. **Archivo:** avance1/ejercicio4_autorizacion.py

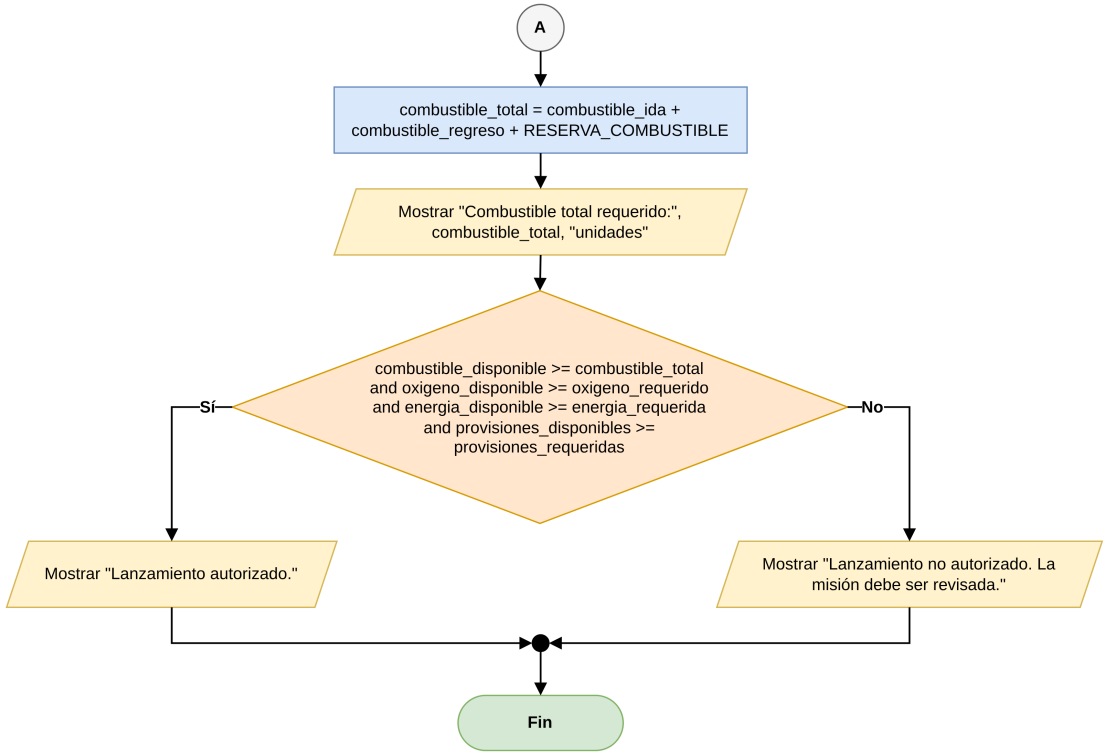
Tabla de variables

Nombre	Tipo de dato	Descripción	Clasificación	Valor inicial / Restricción
RESERVA_COMBUSTIBLE	int	Reserva obligatoria de combustible para emergencias	Constante	10
combustible_disponible	int	Combustible con que cuenta la nave	Entrada	Entero positivo (> 0)
combustible_ida	int	Combustible requerido para llegar al destino	Entrada	Entero positivo (> 0)
combustible_regreso	int	Combustible requerido para regresar	Entrada	Entero positivo (> 0)
oxigeno_disponible	int	Oxígeno con que cuenta la nave	Entrada	Entero positivo (> 0)
oxigeno_requerido	int	Oxígeno necesario	Entrada	Entero positivo (> 0)
energia_disponible	int	Energía con que cuenta la nave	Entrada	Entero positivo (> 0)
energia_requerida	int	Energía necesaria	Entrada	Entero positivo (> 0)
provisiones_disponibles	int	Provisiones con que cuenta la nave	Entrada	Entero positivo (> 0)
provisiones_requeridas	int	Provisiones necesarias	Entrada	Entero positivo (> 0)
combustible_total	int	Ida + regreso + reserva; se compara contra el disponible	Salida	Calculado

Los mensajes "Lanzamiento autorizado." y "Lanzamiento no autorizado. La misión debe ser revisada." son literales de salida mostrados según la decisión.

Diagrama de flujo — Ejercicio 4





Elaborado en draw.io — archivo editable: documentacion/diagramas/diagrama_ej4.drawio

Código fuente — ejercicio4_autorizacion.py

```
# ============================================================
# Archivo: ejercicio4_autorizacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Bryner Moraga Gomez
# Fecha: 12/10/2026    Versión: 1.0
# Descripción: Realiza la verificación general de recursos y
#              autoriza o rechaza el lanzamiento de la misión.
#              Estructura de control: condicional doble (if/else)
#              sin anidación, con condición compuesta.
# ============================================================

# ---------- Declaración de constantes ----------
RESERVA_COMBUSTIBLE = 10    # reserva obligatoria para emergencias

# ---------- Entrada de datos ----------
combustible_disponible = int(input("Ingrese el combustible disponible: "))
combustible_ida = int(input("Ingrese el combustible requerido para llegar al destino: "))
combustible_regreso = int(input("Ingrese el combustible requerido para regresar: "))
oxigeno_disponible = int(input("Ingrese el oxígeno disponible: "))
oxigeno_requerido = int(input("Ingrese el oxígeno requerido: "))
energia_disponible = int(input("Ingrese la energía disponible: "))
energia_requerida = int(input("Ingrese la energía requerida: "))
provisiones_disponibles = int(input("Ingrese las provisiones disponibles: "))
provisiones_requeridas = int(input("Ingrese las provisiones requeridas: "))

# ---------- Procesamiento ----------
# Se compara contra el total (ida + regreso + reserva) para que
# nunca se autorice una misión que solo tiene combustible para llegar.
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

# ---------- Decisión y salida de resultados ----------
# Una sola condición compuesta con "and": basta que un recurso
# falte para rechazar. No se indica cuál recurso provocó el rechazo.
print()
print("Combustible total requerido:", combustible_total, "unidades")
if (combustible_disponible >= combustible_total
        and oxigeno_disponible >= oxigeno_requerido
        and energia_disponible >= energia_requerida
        and provisiones_disponibles >= provisiones_requeridas):
    print("Lanzamiento autorizado.")
else:
    print("Lanzamiento no autorizado. La misión debe ser revisada.")
```

Casos de prueba — Ejercicio 4

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-14	Ejercicio 4	Todos los recursos suficientes	combustible_disponible = 200 combustible_ida = 40 combustible_regreso = 40 oxigeno_disponible = 100 oxigeno_requerido = 65 energia_disponible = 80 energia_requerida = 60 provisiones_disponibles = 50 provisiones_requeridas = 33	Total = 90 Lanzamiento autorizado.	Combustible total requerido: 90 unidades Lanzamiento autorizado.	Exitoso
CP-15	Ejercicio 4	Caso límite: todos exactamente iguales al mínimo	combustible_disponible = 90 combustible_ida = 40 combustible_regreso = 40 oxigeno_disponible = 65 oxigeno_requerido = 65 energia_disponible = 60 energia_requerida = 60 provisiones_disponibles = 33 provisiones_requeridas = 33	Total = 90 Lanzamiento autorizado.	Combustible total requerido: 90 unidades Lanzamiento autorizado.	Exitoso
CP-16	Ejercicio 4	Combustible solo para llegar al destino	combustible_disponible = 50 combustible_ida = 40 combustible_regreso = 40 oxigeno_disponible = 100 oxigeno_requerido = 65 energia_disponible = 80 energia_requerida = 60 provisiones_disponibles = 50 provisiones_requeridas = 33	Total = 90 (50 < 90) Lanzamiento no autorizado. La misión debe ser revisada.	Combustible total requerido: 90 unidades Lanzamiento no autorizado. La misión debe ser revisada.	Exitoso
CP-17	Ejercicio 4	Combustible para ida y regreso, pero sin la reserva	combustible_disponible = 85 combustible_ida = 40 combustible_regreso = 40 oxigeno_disponible = 100 oxigeno_requerido = 65 energia_disponible = 80 energia_requerida = 60 provisiones_disponibles = 50 provisiones_requeridas = 33	Total = 90 (85 < 90) Lanzamiento no autorizado. La misión debe ser revisada.	Combustible total requerido: 90 unidades Lanzamiento no autorizado. La misión debe ser revisada.	Exitoso
CP-18	Ejercicio 4	Combustible suficiente, energía insuficiente	combustible_disponible = 200 combustible_ida = 40 combustible_regreso = 40 oxigeno_disponible = 100 oxigeno_requerido = 65 energia_disponible = 50 energia_requerida = 60 provisiones_disponibles = 50 provisiones_requeridas = 33	Total = 90 Lanzamiento no autorizado. La misión debe ser revisada.	Combustible total requerido: 90 unidades Lanzamiento no autorizado. La misión debe ser revisada.	Exitoso

Evidencia de ejecución

```
Terminal — mision-orbita/avance1
$ python3 ejercicio4_autorizacion.py
Ingrese el combustible disponible: 200
Ingrese el combustible requerido para llegar al destino: 40
Ingrese el combustible requerido para regresar: 40
Ingrese el oxígeno disponible: 100
Ingrese el oxígeno requerido: 65
Ingrese la energía disponible: 80
Ingrese la energía requerida: 60
Ingrese las provisiones disponibles: 50
Ingrese las provisiones requeridas: 33

Combustible total requerido: 90 unidades
Lanzamiento autorizado.
```

CP-14 — Todos los recursos suficientes

```
Terminal — mision-orbita/avance1
$ python3 ejercicio4_autorizacion.py
Ingrese el combustible disponible: 90
Ingrese el combustible requerido para llegar al destino: 40
Ingrese el combustible requerido para regresar: 40
Ingrese el oxígeno disponible: 65
Ingrese el oxígeno requerido: 65
Ingrese la energía disponible: 60
Ingrese la energía requerida: 60
Ingrese las provisiones disponibles: 33
Ingrese las provisiones requeridas: 33

Combustible total requerido: 90 unidades
Lanzamiento autorizado.
```

CP-15 — Caso límite: todos exactamente iguales al mínimo

```
Terminal — mision-orbita/avance1
$ python3 ejercicio4_autorizacion.py
Ingrese el combustible disponible: 50
Ingrese el combustible requerido para llegar al destino: 40
Ingrese el combustible requerido para regresar: 40
Ingrese el oxígeno disponible: 100
Ingrese el oxígeno requerido: 65
Ingrese la energía disponible: 80
Ingrese la energía requerida: 60
Ingrese las provisiones disponibles: 50
Ingrese las provisiones requeridas: 33

Combustible total requerido: 90 unidades
Lanzamiento no autorizado. La misión debe ser revisada.
```

CP-16 — Combustible solo para llegar al destino

```
Terminal — mision-orbita/avance1
$ python3 ejercicio4_autorizacion.py
Ingrese el combustible disponible: 85
Ingrese el combustible requerido para llegar al destino: 40
Ingrese el combustible requerido para regresar: 40
Ingrese el oxígeno disponible: 100
Ingrese el oxígeno requerido: 65
Ingrese la energía disponible: 80
Ingrese la energía requerida: 60
Ingrese las provisiones disponibles: 50
Ingrese las provisiones requeridas: 33

Combustible total requerido: 90 unidades
Lanzamiento no autorizado. La misión debe ser revisada.
```

CP-17 — Combustible para ida y regreso, pero sin la reserva

```
Terminal — mision-orbita/avance1
$ python3 ejercicio4_autorizacion.py
Ingrese el combustible disponible: 200
Ingrese el combustible requerido para llegar al destino: 40
Ingrese el combustible requerido para regresar: 40
Ingrese el oxígeno disponible: 100
Ingrese el oxígeno requerido: 65
Ingrese la energía disponible: 50
Ingrese la energía requerida: 60
Ingrese las provisiones disponibles: 50
Ingrese las provisiones requeridas: 33

Combustible total requerido: 90 unidades
Lanzamiento no autorizado. La misión debe ser revisada.
```

CP-18 — Combustible suficiente, energía insuficiente

Repositorio Git

Enlace: https://github.com/brynermg/Proyecto_Principios_Programacion.git

Clonar: `git clone https://github.com/brynermg/Proyecto_Principios_Programacion.git`

El repositorio contiene README.md, la carpeta avance1/ con los cuatro programas y la carpeta documentacion/ con este documento.